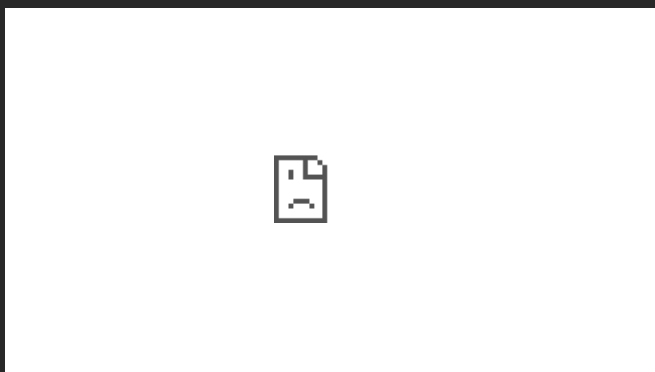


Inigo Quilez :: articles :: outdoors lighting - 2013

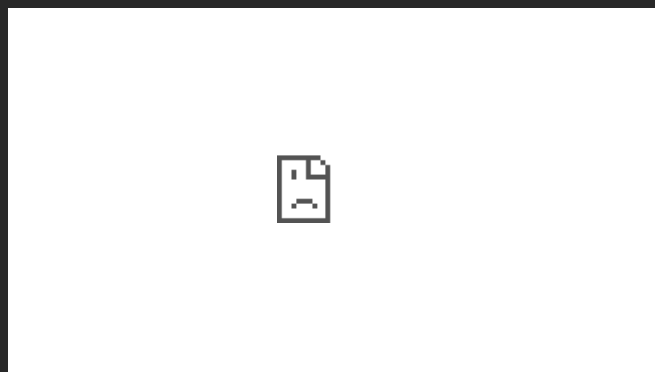


Intro

When doing realtime rendering in the form of small demos and experiments you normally cannot afford complex lighting systems (baked irradiance, cone tracing in voxelized scenes, whatever you are into). However, in those cases you can sometimes achieve convincing images by using lights cleverly, especially when doing outdoors lighting of big landscapes, where the contribution of indirect lighting is just moderate and predictable.



Same three lights rig (click "play" to see it moving in realtime)



Three lights in a realtime procedural landscape (click "play" to see it moving in realtime)

This articles describes the lighting rig I use when doing such tiny computer graphics experiments with landscapes. It's basically made of 3 or 4 directional lights, one shadow, some (fake or screen space) ambient occlusion, and a fog layer. These few elements tend to behave nicely and even look fotoreal-ish if balanced properly. Of course I don't claim this to be the best way to do exterior lighting, but it seems to be working fine for me and it is usually the starting point for more complex rigs.

The color space

The main thing before you start doing any work in lighting and shading at all, is to make sure you are working in a linear color/lighting space. This means you are doing your lighting, shading and color maths as usual, but that you do have a gamma curve applied too, applied at the very end of the process right before display. Don't even bother trying to do tweak lights, materials, diffuse falloff shapes or anything if you don't have the gamma correction curve in the end of your rendering pipeline. Really...

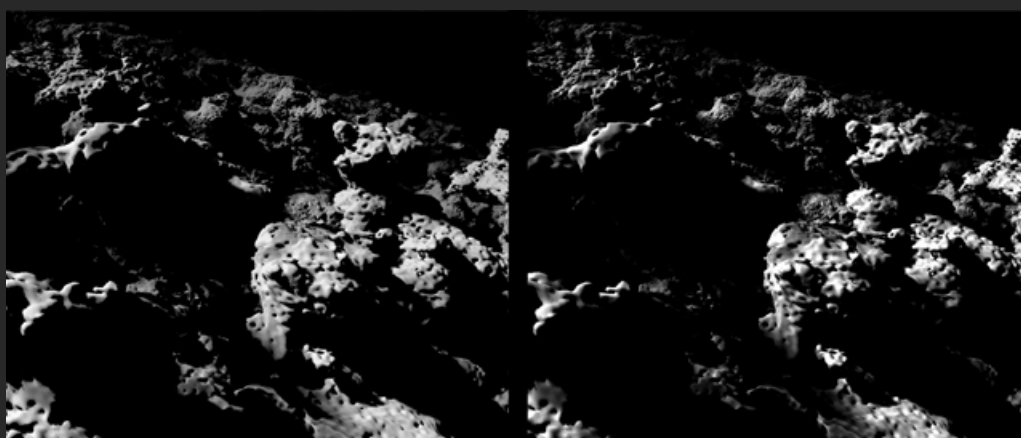
```
// do amazing visuals
vec3 color = doFantasticDemo();

// gamma correction
color = pow( color, vec3(1.0/2.2) );

// final step, display (and perhaps color grade)
displayColor = color;
```

If you are coming from a non-linear world, like if you are porting an old demo or game, and use a gamma corrected pipeline, you'll notice your colors are much brighter now and that you lost contrast. Don't panic and DO NOT TOUCH the gamma correction curve, leave it at 1.0/2.2. Instead, go and adjust your lights. Reduce then ambient, fill lights, the indirect lighting. Doing so will bring the values in your lighting rig to a more real/physical ranges. You'll soon learn that an ambient/fill should in fact be pretty small compared to the key light.

One of the benefits of working in a linear space is that you no longer have to tweak the diffuse falloff in order to get realistic gradients of light in your Lambert shaders. The gamma curve is doing it for you, so your diffuse BRDF can be just plain N.L as theory predicts. (perhaps you have never tweaked diffuse lighting before, but a way I've been using to achieve more realistic images before was to use $\sqrt{\max(\text{dot}(\mathbf{N}, \mathbf{L}), 0)}$, which I don't need anymore)



Left, diffuse lighting component in linear space + gamma correction. Right, non linear space (no gamma correction).

Materials

Besides the color space, there's another thing you need to lock to the right value or range before you start doing lighting: your materials.

Without going into detail of physically correct specular BRDFs or anything, and focusing only in the diffuse component, make sure your diffuse colors are around 0.2, and no much brighter except for very special situations.

If you are working in a vanilla framework without gamma or anything, you might be tempted to think of materials values and colors as representative of the final color they'll have in the screen. That output-driven approach will get you more in trouble than anything, for the material's color/intensity/value represent only the amount of light they reflect. So, keep your materials/textures/colors in the range of 0.2, and if you need to make your objects brighter in screen then what you want to do is to make the lights more intense. Not the materials.

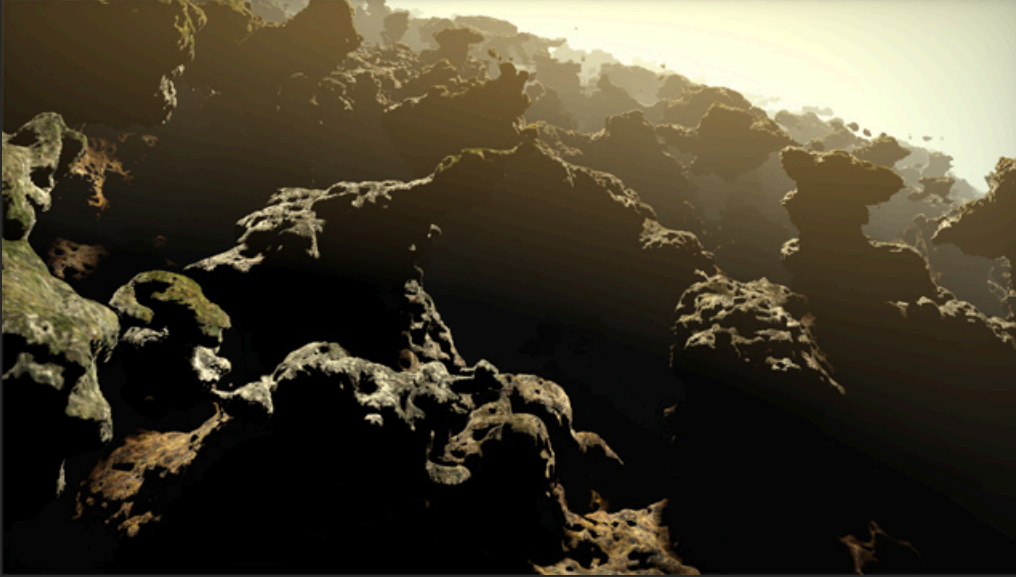
This becomes specially important in global illumination setups, where the diffuse color of the materials will be responsible for most of the ambient light bouncing around in the scene and hence the overall contrast of the image. If you set the materials colors too high, there will be too much indirect light floating around. Even without global illumination, like in these tutorial where we are going to fake the indirect lighting with manually places bounce lights, doing the approximately correct thing is still desirable.

Borrom line again - make your materials no bigger than 0.2 as a rule, and you'll be safe. If you need a brighter image, make the lights brighter, not the materials.

First light

We are going to through our first light to the scene. Some poeple find it might be easier to hit your look by starting to tweak the keylight or the fill lights first, depending wether the image is being exposed for the sun or the shadows. If you have implemented automatic exposure/tonemapping, then it doesn't really matter, as long as the values are correct in relation to each other. In my case I have not automatic tonemapping in place in

this tiny rendering experiments, and I usually find images prettier when exposed for the shadows. However I'll start with the key light, just because:



The light from the sun as keylight.

In these kind of open outdoors scenes, the keylight is clearly the sun. You naturally want to make it yellowish and pretty intense, around 1.0 or 1.5. Since the albedo of the materials are around 0.2, and there's the gamma correction curve, right now the overall color of a lit pixel is approximately 0.5, or 128 in an 8 bit buffer, since $\text{pow}(0.2 * 1.25, 1.0/2.2)$ is 0.53. This is a great start!

Since in this case our key light is the sun, we really want to do some shadowing and capture the important features of the shape of the terrain in the image. Ideally you want to make this shadows a little bit soft. Because this particular example image I'm using in this article is based on raymarching, I can super easily achieve some soft shadow effect for free by casting one single ray. See my other [article on soft shadows](#) to see how to do that.

There's one great trick regarding shadows that works fantastic and helps make beautiful sunset or sunrise scenes - and that's to colorize and over saturate the penumbra of the shadows into some red or orange color. You can achieve that by promoting your penumbra scalar signal into a color and then power-ing it:

```
// compute soft shadows
float shadow = doGreatSoftShadow( pos, sunDir );

// colorize penumbras
vec3 cshadow = pow( vec3(shadow), vec3(1.0, 1.2, 1.5) );
```

Another very important tip is to avoid using (ambient) occlusion for the key light. I've seen many demos and games fall into this error, and it really makes things ugly and horribly CG looking. The ambient occlusion should be reserved for the fill lights. The sun, being such a small source (in solid angle), is pretty much visible or invisible, and the shadow ray/map is taking care of this already. So please, do not modulate the sun light with ambient occlusion!!

Second light

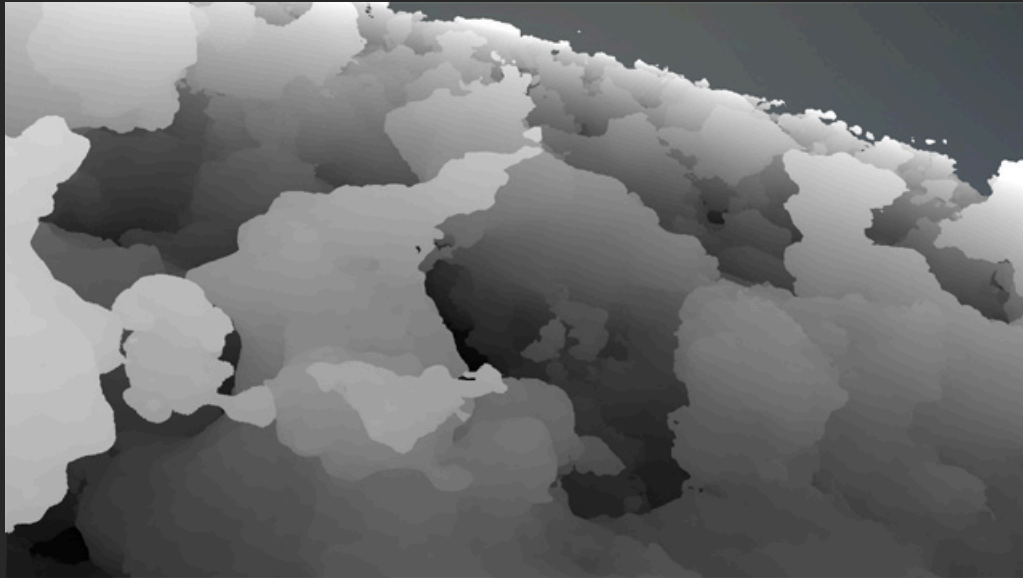
The second light source we have to take care of is the sky itself. Of course this will be blue-ish in color, and not very bright compared to the key light. A value of 0.2 works alright.



Sky light as secondary source

In theory we should be firing several rays in the hemisphere at the shading point and gathering `visibility_times_skycolor` with them. However, since this can be very expensive, a good compromise works fine for me in many cases is to replace the sky dome with a single directional light falling straight vertically on the set. You can colorize the light based on the angle of incidence if you need to create sunset sky lighting.

Also, because we are no longer integrating `visibility_times_color`, this is a good change to use some cheap occlusion computation as replacement. If you are working in a rasterizer, probably some screen space occlusion or even some **multiresolution ambient occlusion** can work. If you are in the context of raymarching, you can simply cast a vertical shadow ray, or use some other cheap occlusion technique. A last alternative is to use some procedural occlusion signal extracted from the geometry itself.



Procedural occlusion

Third light

The last light of our rig is used to implement indirect lighting, without doing global illumination. Since the main source of indirect light is the bounce of the sun light into the scene itself being reflected back in the opposite direction it was coming from (in overall), we can simply put a third and last directional light coming from approximately the opposite direction of the sun. This light we will make it horizontal, so we are basically making copying the two horizontal coordinates of the sun direction and negating them, and leaving the vertical dimension to zero.



Indirect light added to the scene

The color of this light should be approximately what you would get after bouncing the sun light, which was intensity 1.25 with the diffuse component of the materials in the scene, which we made 0.2. So the intensity of this indirect light should be approximately 0.3 in value, and be colored as "yellow times scene". It's not a bad idea to give this light some extra saturation, though.

As for visibility, there's no good answer for this one, but in general since indirect lighting has to be gathered in the hemisphere, using the same ambient occlusion signal used for the sky dome works okeish in this case as well.

All these lights together give as the following lighting equation so far:

```
// compute materials
vec3 material = doFantasticMaterialColor( pos, nor );

// lighting terms
float occ = doGorgeousOcclusion( pos, nor );
float sha = doGreatSoftShadow( pos, sunDir );
float sun = clamp( dot( nor, sunDir ), 0.0, 1.0 );
float sky = clamp( 0.5 + 0.5*nor.y, 0.0 1.0 );
float ind = clamp( dot( nor, normalize(sunDir*vec3(-1.0,0.0,-1.0)) ), 0.0, 1.0 );

// compute lighting
vec3 lin = sun*vec3(1.64,1.27,0.99)*pow(vec3(sha),vec3(1.0,1.2,1.5));
        lin += sky*vec3(0.16,0.20,0.28)*occ;
        lin += ind*vec3(0.40,0.28,0.20)*occ;

// multiply lighting and materials
vec3 color = material * lin;

// apply fog
color = doWonderfullFog( color, pos );
```

```
// gamma correction
color = pow( color, vec3(1.0/2.2) );

// display
displayColor = color;
```

Speaking of the fog, it's always a bad idea not to use vanilla constant-color fog, but use some form of **more color rich fog**. There's no reason not to!

This is how the rig looks like from different points of view:

Video unavailable

This video is unavailable

inigo quilez - learning computer graphics since 1994